League of Legends Win Prediction

INFO-6150: Data Mining and Analysis

Professor Shweta Pathak

Andrew P., Bruno M., Jadd A.

Fanshawe College AIM1/2

April 14, 2026

TABLE OF CONTENTS

1. OBJECTIVE

This report examines the outcome of League of Legend (LoL) matches. Specifically, how certain features, such as character selection and player role, correlate with victory/defeat and how they even correlate between themselves.

The company of LoL, Riot Games, has made available many of there in-game statistics available through there APIs. Through this, we will examine selected features with Python and Machine Learning (ML), then illustrate our evidence with Power BI.

2. DATA

2.1 Why This Dataset?

We chose LoL as our report subject for two reasons:

1. With LoL being one the world's most popular game since 2010, there are billions of matches played and millions of players.
2. We also have extensive experience in LoL and gaming in general.

LoL's massiveness and our understanding allows to identify and probe the nuanced features for excellent ML modelling.

2.2 Descriptions

<table>
<tr><td>Fetch Parameters
Username
Tag</td><td>Player Statistics
Champion (configuration)
Lane (configuration)
Gold (resource)
Kill/Death/Assist[KDA] (performance)</td></tr>
<tr><td colspan="2">Team Statistics
If Victorious (target)
Counts and Timings of Dragon and Baron Completion (objectives)
Total Amount of Gold of Team Players (resource)
Average, Counts, and Timings of team player’s KDA (performance)</td></tr>
</table>

2.3 API

The API we used is the official League of Legends client for account data in the Americas. To get the data, we fetched some players account data by username parameters. With these accounts, we extracted a sample of their games. Since LoL is such a dominant game, this operation was very expensive. Many accounts have many games with many people accessing the same API. So, we had to abide by regulation rules like using timed keys and request timeouts.

2.4 Transformations

1. We first reduced the data by removing unwanted matches such as extra game modes and unfinished games.
2. Many of the values were already encoded so the tricky parts next are to unravel lists so our model can read statistics at full.
3. We further parsed the data by splitting the two teams. Each team has its own unique statistics such as resource counts, objective timings, and if victorious.
4. Within the team, we also parsed the individual stats of the five players on each team. Each player also has unique statistics such as character selection, resources, and performance.
5. Now we have all the statistics parsed, we will build the dataframe by having each row represent one team's match with all team and its player statistics mentioned.

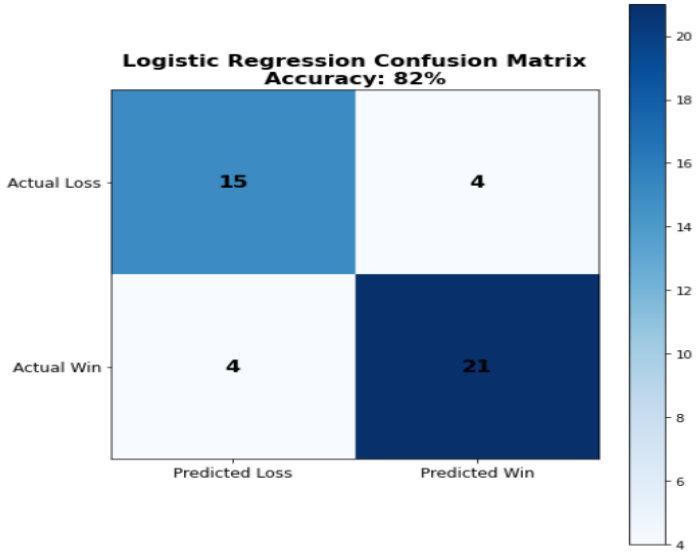
3. MACHINE LEARNING

Our machine learning method was logistic regression. This is because we are doing a binary classification, whether the team will win or not. This model is fed various statistics mentioned before, such as resources, performance, timings, and frequency, then learns whether those statistics influence the binary outcome using linear relations.

4. ANALYSIS

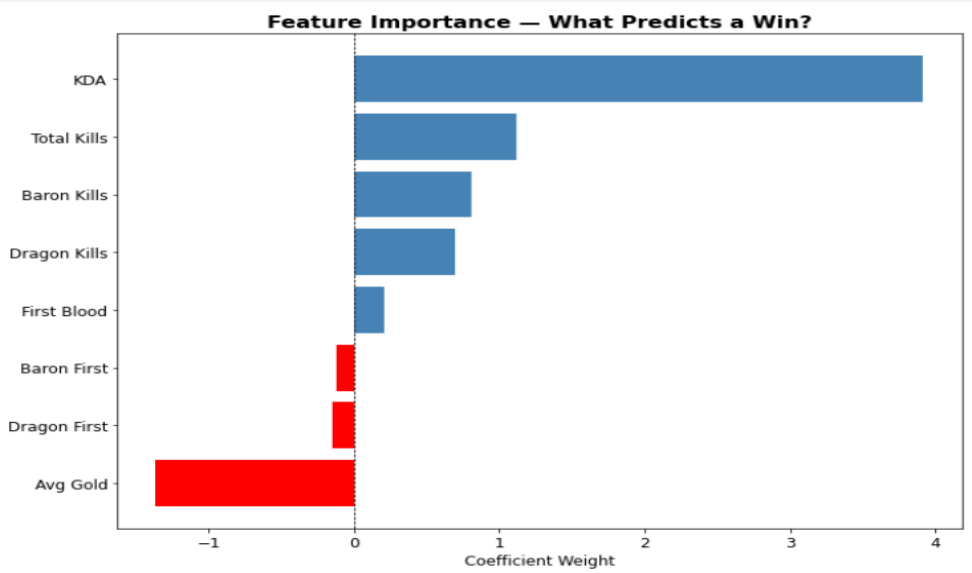
4.1 Model Evaluation

From the graph below, the model was a success at 82% accuracy. If the dataset was imbalanced, such as having much more win or loses, accuracy wouldn't be a good measure. However, since the games we fetched are split by win and lose, it's exact 50/50.



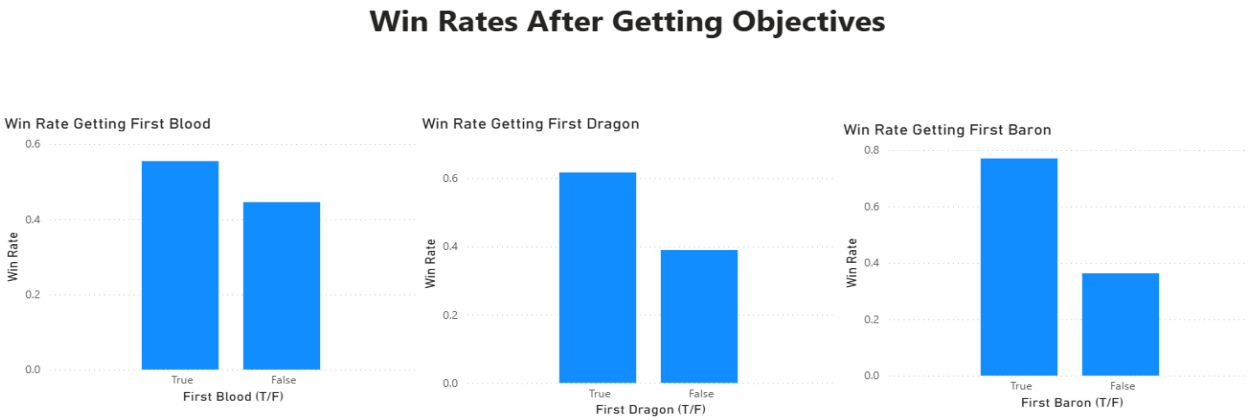
4.2 Feature Importance

The blue bars represent features that influence wins while red mean otherwise. The size of the bars is how much those features sway the outcome. Then number of "Kills" seem to heavily influence winning while low averages of gold strongly decrease win probability.



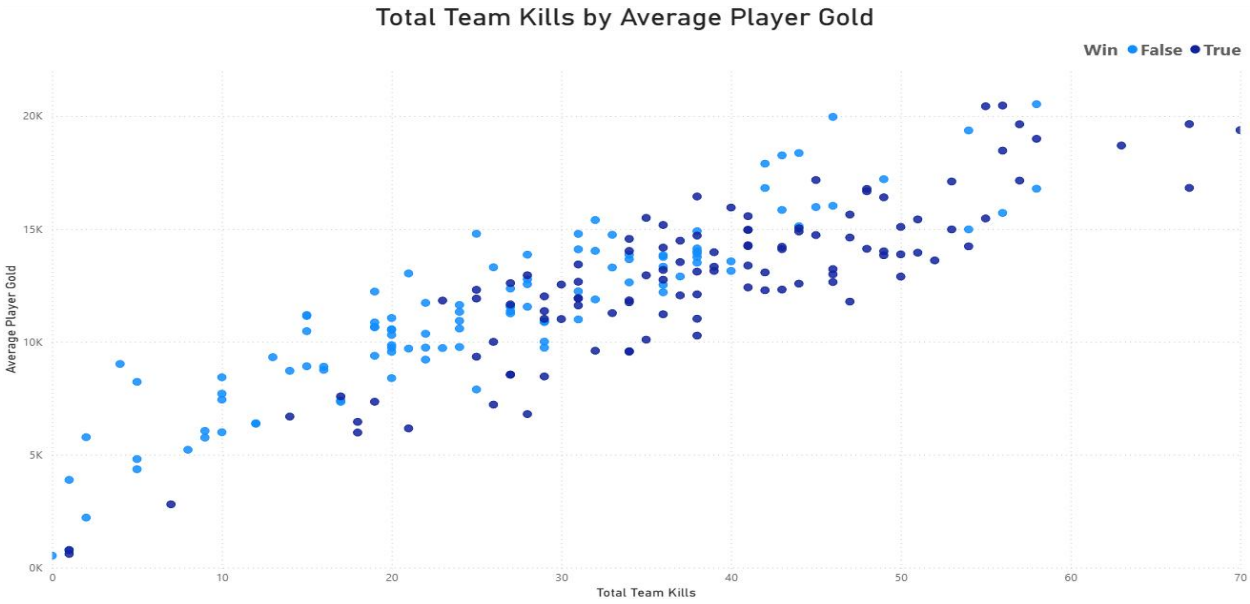
4.3 Storytelling | Early Action

There's evidence that getting "First blood" and objectives first such as Dragon and Baron pushes winning probability. Those who go first have their win% up significantly as seen with "Baron First" as it goes from 40% to 80%. Stakeholders such as players or coaches can use these key points to gain the upper hand.



4.4 Storytelling | Getting Resources

There is a strong positive correlation in getting "Kills" and getting gold. However, the correlation between those two statistics and winning aren't as correlated. We see that teams with minimal gold and "Kills" are mostly lost games while only those in the high-end are wins. Yet, the middle ground is a cluster of inseparable wins and losses meaning game with teams with that gold/kill range are still competitive.



4.5 ART | Objective Importance

Here is a visual outlining how getting objectives increases win probability. For those who play, we can see those who get either Baron or Dragon, which are located northwest and southeast of the center and highlighted by circles, have an astonishing +60% win rate. However, those who get both seem to spread their manpower thin and have a lower win rate at 58%. Getting neither objective means the team loses special in-game perks which lowers where win rate at 55%. Since all these win rates are still favourable at +55%, we know that these objectives only help but don't provide any near certainty regarding the outcome.

5. Conclusion

Using a logistic regression to predict wins or losses, it's evident that many features influence the outcome, but the extremes values can only push predictions to near certainty. As seen with the "Early Action" section, completing team objectives first has a strong influence, yet the late team still always has a chance at around 40%. We saw that although "Kills" give more gold it doesn't cement win/loss unless extreme. Many teams with moderate amounts of gold and kills are still competitive to win.

6. Resources

Riot Developer Portal. (n.d.). Developer.riotgames.com. https://developer.riotgames.com/apis

7. M-CODE

```
import requests
import pandas as pd
import time

key = "RGAPI-56340e80-ebe0-460d-8e54-46c2dfa48a44"
url = "https://americas.api.riotgames.com"
numberOfMatches = 10

players = [
   {"name": "calamitycucco", "tag": "NA1"},
   {"name": "Scooby Stacks", "tag": "Stack"},
   {"name": "Allen Clone", "tag": "NA1"},
   {"name": "daza121", "tag": "NA1"},
   {"name": "fee6", "tag": "NA1"},
   {"name": "LoopyLegend", "tag": "NA1"},
   {"name": "Akanos", "tag": "NA1"},
   {"name": "Haast", "tag": "NA1"},
   {"name": "LeWork", "tag": "NA1"},
   {"name": "Finan", "tag": "MTQ"},
   {"name": "LeGroose", "tag": "NA1"},
   {"name": "Yooniversal", "tag": "yuna9"},
```

```
    {"name": "PrinceTarari", "tag": "NA1"},
    {"name": "PiggyTofu", "tag": "NA1"},
    {"name": "HailTheBeast", "tag": "kled"}
]

# Get player PUUIDs
for i in range(len(players)):
    time.sleep(1.2)
    response = requests.get(
        url + f"/riot/account/v1/accounts/by-riot-
id/{players[i]['name']}/{players[i]['tag']}?api_key={key}"
    )
    data = response.json()
    if "puuid" in data:
        players[i]["puuid"] = data["puuid"]
    else:
        print(f"FAILED: {players[i]['name']}#{players[i]['tag']} -> {data}")
        players[i]["puuid"] = None

# Filter out players with no puuid
players = [p for p in players if p["puuid"] is not None]

# Get match IDs
allMatchesIds = []
for i in range(len(players)):
    time.sleep(1.2)
    response = requests.get(
        url + f"/lol/match/v5/matches/by-
puuid/{players[i]['puuid']}/ids?&start=0&count={numberOfMatches}&api_key={
key}"
    )
    for item in response.json():
        allMatchesIds.append(item)

allMatchesIds = list(set(allMatchesIds))

# Get match details
```

```
allMatches = []
for id in allMatchesIds:
    time.sleep(1.2)
    response = requests.get(url + f"/lol/match/v5/matches/{id}?api_key={key}")
    data = response.json()

    # Check 1: Skip non-classic games
    if data["info"]["gameMode"] != "CLASSIC":
        continue

    # Check 2: Skip remakes - neither team got first blood
    team1_firstblood =
data["info"]["teams"][0]["objectives"]["champion"]["first"]
    team2_firstblood =
data["info"]["teams"][1]["objectives"]["champion"]["first"]
    if not team1_firstblood and not team2_firstblood:
        continue

    match = {}
    match["gameMode"] = data["info"]["gameMode"]
    participants = data["info"]["participants"]

    playersInMatch = []
    for participant in participants:
        player = {
            "champion": participant["championName"],
            "lane": participant["individualPosition"],
            "win": participant["win"],
            "goldEarned": participant["goldEarned"],
            "kills": participant["kills"],
            "deaths": participant["deaths"],
            "assists": participant["assists"]
        }
        playersInMatch.append(player)

    match["team1"] = {"players": playersInMatch[:5]}
    match["team2"] = {"players": playersInMatch[5:]}
```

```
    match["team1"]["objectives"] = {"dragon": {}, "baron": {}, "champion": {}}
    match["team2"]["objectives"] = {"dragon": {}, "baron": {}, "champion": {}}

    match["team1"]["objectives"]["dragon"] =
data["info"]["teams"][0]["objectives"]["dragon"]
    match["team2"]["objectives"]["dragon"] =
data["info"]["teams"][1]["objectives"]["dragon"]
    match["team1"]["objectives"]["baron"] =
data["info"]["teams"][0]["objectives"]["baron"]
    match["team2"]["objectives"]["baron"] =
data["info"]["teams"][1]["objectives"]["baron"]
    match["team1"]["objectives"]["champion"] =
data["info"]["teams"][0]["objectives"]["champion"]
    match["team2"]["objectives"]["champion"] =
data["info"]["teams"][1]["objectives"]["champion"]

    allMatches.append(match)

# Build restructured rows - one per team per match
allRows = []

for match in allMatches:
    for teamNum, teamKey in enumerate(["team1", "team2"], start=1):
        team = match[teamKey]
        row = {}
        row["gameMode"] = match["gameMode"]
        row["team"] = teamNum
        row["baronFirst"] = team["objectives"]["baron"]["first"]
        row["baronKills"] = team["objectives"]["baron"]["kills"]
        row["dragonFirst"] = team["objectives"]["dragon"]["first"]
        row["dragonKills"] = team["objectives"]["dragon"]["kills"]
        row["championFirst"] = team["objectives"]["champion"]["first"]
        row["championKills"] = team["objectives"]["champion"]["kills"]
        row["win"] = team["players"][0]["win"]
        row["totalGold"] = sum(p["goldEarned"] for p in team["players"])
        row["totalKills"] = sum(p["kills"] for p in team["players"])
```

```
        row["totalDeaths"] = sum(p["deaths"] for p in team["players"])
        row["totalAssists"] = sum(p["assists"] for p in team["players"])
        row["avgGold"] = row["totalGold"] / 5
        row["kda"] = (row["totalKills"] + row["totalAssists"]) /
max(row["totalDeaths"], 1)

        # Individual player stats
        for i, player in enumerate(team["players"]):
            row[f"player{i}.champion"] = player["champion"]
            row[f"player{i}.lane"] = player["lane"]
            row[f"player{i}.kills"] = player["kills"]
            row[f"player{i}.deaths"] = player["deaths"]
            row[f"player{i}.assists"] = player["assists"]
            row[f"player{i}.goldEarned"] = player["goldEarned"]

        allRows.append(row)

df = pd.DataFrame(allRows)
```